GRUPRO — Segment Trees

introduction, lazy propagation

Jonathan Queiroz

Motivação

Range minimum query

Apresentação do problema

- Considere um vetor $A = \{a_1, a_1, \cdots, a_n\}$
- Operações:
  - `update(i, x)`: modifica o valor de a_i para x.
  - `query(i, j)`: retorna $\min\{a_i, a_{i+1}, \ldots, a_j\}$.

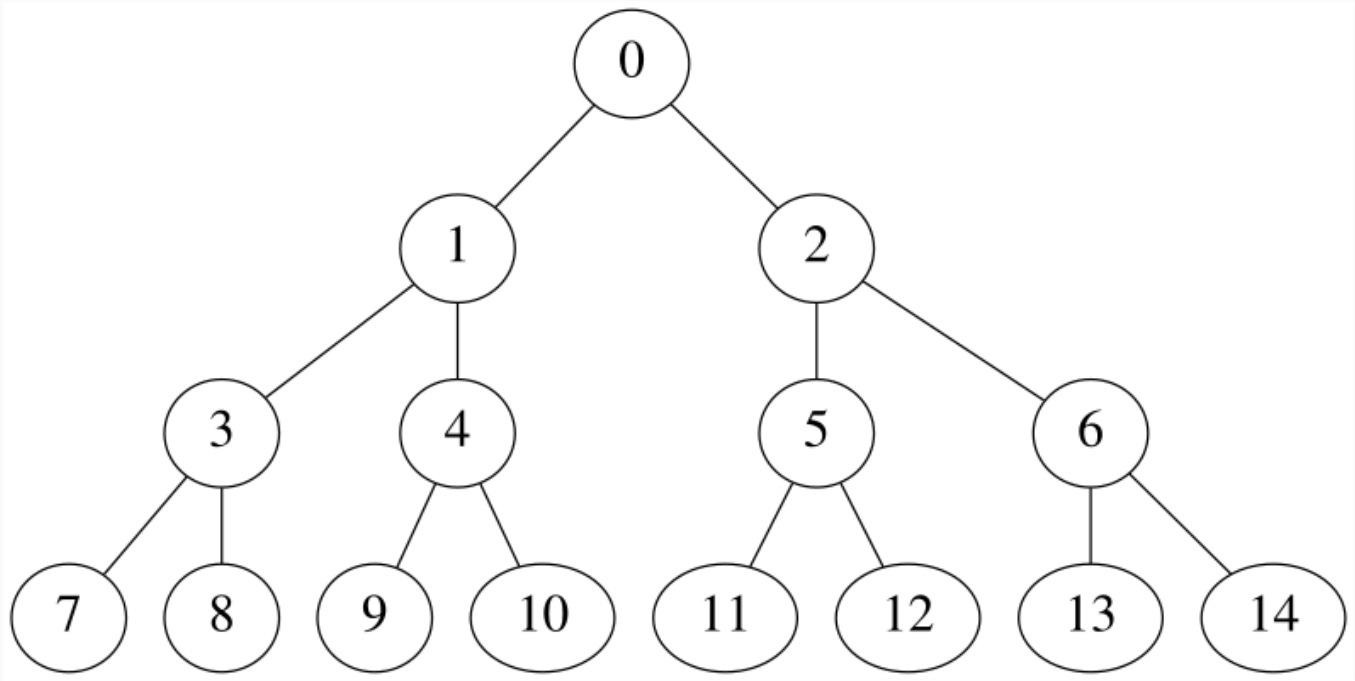
Solução ingênua

```
void update(int i, int x) {
    v[i] = x;
}
int query(int l, int r) {
    int ans = v[l];
    for (int i = l; i <= r; ++i)
        ans = min(ans, v[i]);
    return ans;
}
```

Conceitos básicos

Representação de árvore em vetor

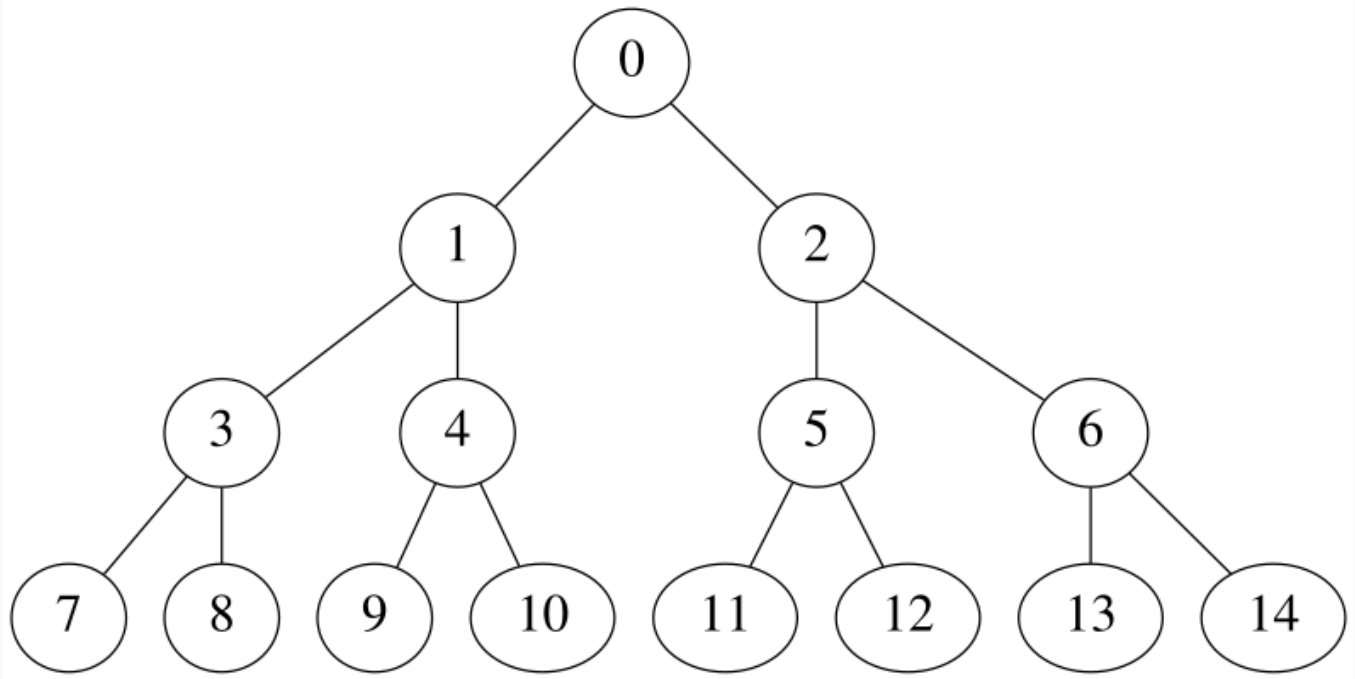
- Precisamos atribuir um índice distinto para cada nó
  - raiz $= 0$
  - filho-esquerdo$(x) = 2x + 1$
  - filho-direito$(x) = 2x + 2$



Conceitos básicos

Propriedades

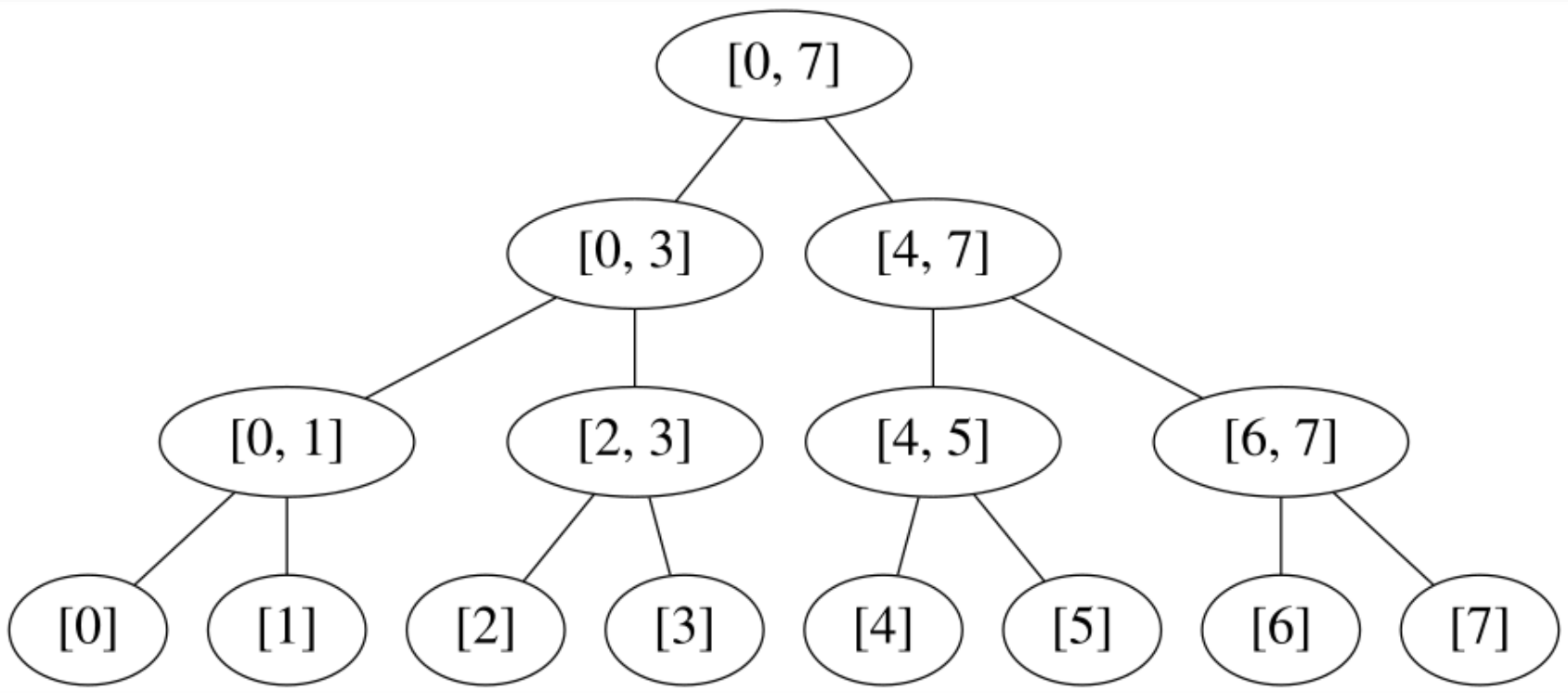
- Uma árvore binária completa com N folhas apresenta $2N - 1$ nós
- Uma árvore binária completa com N folhas apresenta $\log_2 N + 1$ níveis



Árvore de segmentos

Ideia

- Cada nó guarda o valor mínimo de um intervalo contíguo do vetor original A



Implementação (build)

```
int tree[MAX_N*4];

void build(int no, int esq, int dir) {
    if (esq == dir) {
        tree[no] = v[esq];
    } else {
        int meio = (esq + dir) / 2;
        build(2*no+1, esq, meio);
        build(2*no+2, meio+1, dir);
        tree[no] = min(tree[2*no+1], tree[2*no+2]);
    }
}
```

Árvore de segmentos

Implementação (query)

```
int query(int no, int esq, int dir, int i, int j) {
  if (esq == i && dir == j)
    return tree[no];

  int meio = (esq + dir) / 2;
  if (j <= meio)
    return query(2*no+1, esq, meio, i, j);
  else if (i > meio)
    return query(2*no+2, meio+1, dir, i, j);
  else
    return min(query(2*no+1, esq, meio, i, meio),
               query(2*no+2, meio+1, dir, meio+1, j));
}
```

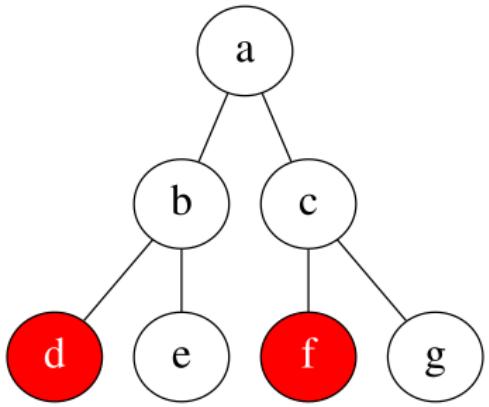
Implementação (update)

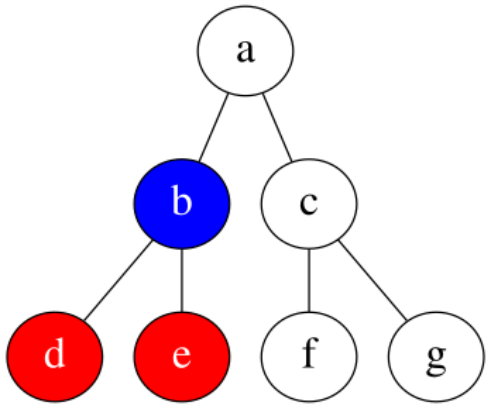
```
void update(int no, int esq, int dir, int i, int x) {
    if (esq == dir) {
        tree[no] = x;
    } else {
        int meio = (esq + dir) / 2;
        if (i <= meio)
            update(2*no+1, esq, meio, i, x);
        else
            update(2*no+2, meio+1, dir, i, x);
        tree[no] = min(tree[2*no+1], tree[2*no+2]);
    }
}
```

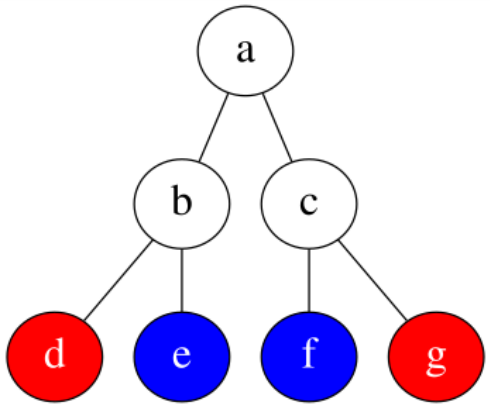
Análise

Complexidade

- Construção: $\Theta(N)$
  - Cada nó da árvore é acessado precisamente uma vez.
- Modificação: $\Theta(\log N)$
  - Precisamente um nó de cada nível é acessado.
- Consulta: $\Theta(\log N)$
  - No máximo dois nós de cada nível são acessados.
  - Como verificar isso?







Generalizações

- Ao invés de mínimo: qualquer operação associativa
  - Ex: máximo, soma, produto, etc.
- Ao invés de inteiros: tipos arbitrários
  - Ex: `std::vector`, `std::set`, structs, etc.

Desafio 1

- Consulta: qual a maior soma de prefixo do intervalo [i, j]?
  - $\max\{a_i, a_i + a_{i+1}, a_i + a_{i+1} + a_{i+2}, \ldots, a_i + a_{i+1} + \cdots + a_j\}$

```
struct Data {
    int maximum_prefix_sum;
    int sum;
};
```

Desafio 2 (SPOJ-GSS3)

- Consulta: qual a maior soma de subvetor do intervalo [i, j]?

Lazy propagation

Operações

- `update(i, j, x)`: acrescenta x a $a_i, a_{i+1}, \ldots, a_j$.
- `query(i, j)`: retorna $a_i + a_{i+1} + \cdots + a_j$.

Implementação (build)

```
int tree[MAX_N*4];
int lazy[MAX_N*4];
void build(int no, int esq, int dir) {
    if (esq == dir) {
        tree[no] = v[esq];
        lazy[no] = 0;
    } else {
        int meio = (esq + dir) / 2;
        build(2*no+1, esq, meio);
        build(2*no+2, meio+1, dir);
        tree[no] = tree[2*no+1] + tree[2*no+2];
        lazy[no] = 0;
    }
}
```

Lazy propagation

Implementação (propagate)

```
void propagate(int no, int esq, int dir) {
    if (lazy[no] != 0) {
        if (esq != dir) {
            lazy[2*no+1] += lazy[no];
            lazy[2*no+2] += lazy[no];
        }
        tree[no] += (dir-esq+1) * lazy[no];
        lazy[no] = 0;
    }
}
```

Lazy propagation

Implementação (query)

```
int query(int no, int esq, int dir, int i, int j) {
    propagate(no, esq, dir);
    if (esq == i && dir == j)
      return tree[no];

    int meio = (esq + dir) / 2;
    if (j <= meio)
        return query(2*no+1, esq, meio, i, j);
    else if (i > meio)
        return query(2*no+2, meio+1, dir, i, j);
    else
        return query(2*no+1, esq, meio, i, meio) +
               query(2*no+2, meio+1, dir, meio+1, j);
}
```

Lazy propagation

Implementação (update)

```
void update(int no, int esq, int dir, int i, int j, int x) {
    if (esq == i && dir == j) {
        lazy[no] += x;
    } else {
        int meio = (esq + dir) / 2;
        if (j <= meio) {
            update(2*no+1, esq, meio, i, j, x);
        } else if (i > meio) {
            update(2*no+2, meio+1, dir, i, j, x);
        } else {
            update(2*no+1, esq, meio, i, meio, x);
            update(2*no+2, meio+1, dir, meio+1, j, x);
        }
        propagate(no, esq, dir);
        propagate(2*no+1, esq, meio);
        propagate(2*no+2, meio+1, dir);
        tree[no] = tree[2*no+1] + tree[2*no+2];
    }
}
```

Exercícios

Exercícios (links clicáveis)

Competição no codepit

- Link: http://goo.gl/oSGUao

Diretos

- Perguntas de Mínimo (CF)
  - necessário entrar no grupo do GRUPRO no codeforces (clique aqui)
- Distribuição de Gorgonzola (CF)
  - necessário entrar no grupo do GRUPRO no codeforces (clique aqui)

Desafios

- Computação Teórica (CF)
  - necessário entrar no grupo do GRUPRO no codeforces (clique aqui)
- Maximum Sum (SPOJ-KGSS, codepit)
- Can you answer these queries III (SPOJ-GSS3, codepit)

Lazy propagation

- Horrible Queries (SPOJ-HORRIBLE, codepit)
- Multiples of 3 (SPOJ-MULTQ3, codepit)
- Sum of Squares with Segment Tree (SPOJ-SEGSQRSS, codepit)
- Can you answer these queries IV (SPOJ-GSS4, codepit)

fim

fim